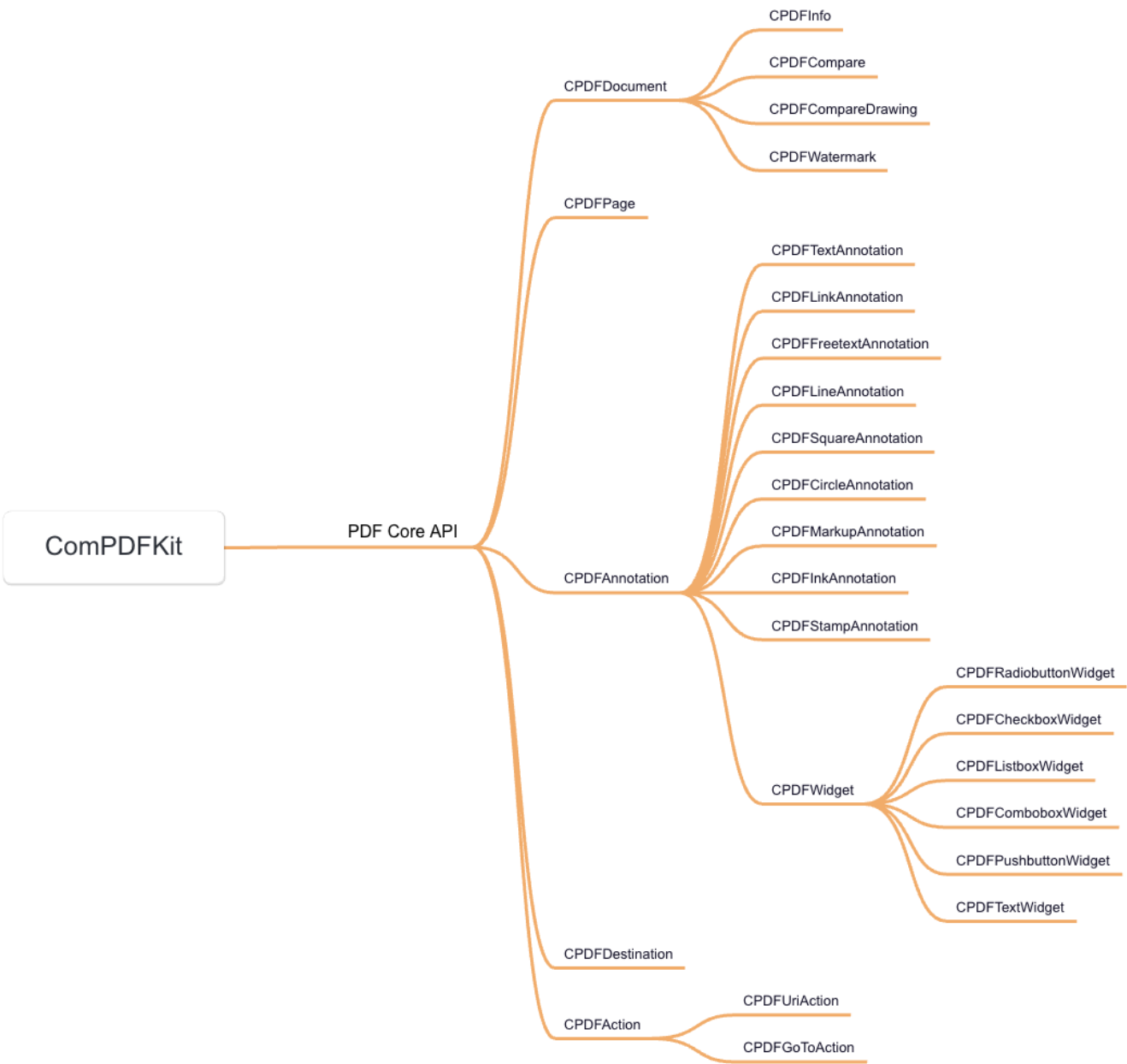


1 概述

ComPDFKit PDF SDK for Java 是一个功能强大的 PDF 库，可供开发人员向其安卓应用程序添加强大的 PDF 功能，它附带简单易用的 Java API，可提供文档创建、查看、批注、编辑、对比、密文标记和表单功能。ComPDFKit PDF SDK 功能丰富且经过数年的迭代优化，即使是 PDF 知识有限的开发者也可以在 Java 平台上通过几行代码快速构建一个专业的 PDF 查看器。

1.1 ComPDFKit PDF SDK

ComPDFKit PDF SDK 组成，如下图所示：



ComPDFKit PDF SDK :

- **PDF Core API**

Core API 可以独立使用，支持文档渲染、分析、文本提取、文本搜索、表单创建与填充、文档加密、文档拆分合并、注释创建和回复等一站式 PDF 功能。但不包括任何 UI 组件。

1.2 主要功能

注释 (Annotations) 组件提供以下功能：

- 创建、编辑、删除和回复注释，包括注解、链接、文字、直线、矩形、圆形、亮高、下划线、波浪线、删除线、图章等注释类型。
- 支持注释外观调整。
- 导入和导出 XFDF 格式的注释数据。
- 支持注释展平。
- 预定义注释样式。

表单 (Forms) 组件提供以下功能：

- 创建、编辑和删除表单字段，包括按钮、复选框、单选按钮、文本字段、下拉框、列表框。
- 填写 PDF 表单。
- 支持 PDF 表单展平。

文档编辑器 (Document Editor) 组件提供以下功能：

- PDF 操作，包括拆分、提取和合并 PDF 文件。
- 页面编辑，包括删除页面、插入页面、裁剪页面、移动页面、旋转页面、交换页面位置 and 不同文档的页面替换。
- 设置文档信息。
- 提取 PDF 文件图片。

安全 (Security) 组件提供以下功能：

- 加密和解密 PDF，包括权限设置和密码保护。

水印 (Watermark) 组件提供以下功能：

- 创建、编辑、添加、删除、更新、获取水印。
- 支持文字和图片水印。

2 入门指南

只需几行 Java 代码，即可轻松将 ComPDFKit 集成到您的 Java 应用程序中。花几分钟查看本指南并快速开始吧！

以下部分介绍了 ComPDFKit PDF SDK 包的结构、运行的系统要求、如何运行演示和如何使用 ComPDFKit PDF SDK 用 Java 制作 PDF 阅读器。

2.1 系统要求

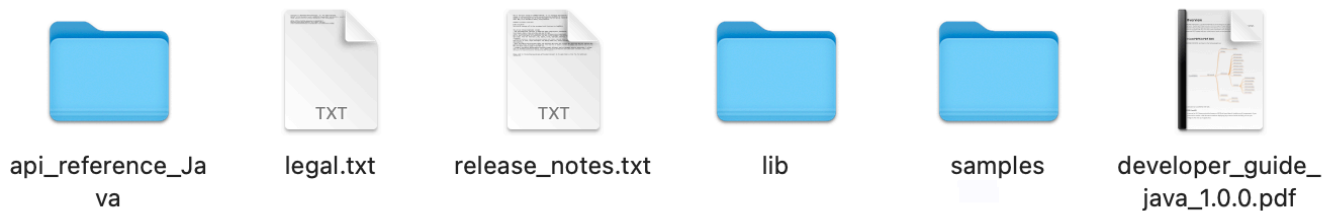
ComPDFKit PDF SDK for Java 需要应用程序启用 Java 8 语言功能才能构建。

- 编程环境: Java JDK 1.8 和更高。
- 支持的平台
 - Mac (intel & M1)
 - Linux (x86)

2.2 Java 包结构

您可以[联系我们](#)获取我们的 PDF SDK 安装包。ComPDFKit PDF SDK for Java 的软件包包括以下文件：

- **"samples"** - 包含Java示例项目的文件夹。
 - **"Annotations"** - 具有完整类型注释编辑的PDF查看器，包括添加注释、修改注释、注释列表等。
 - **"Forms"** - 具有完整类型表单编辑的PDF查看器，包括单选按钮、组合框等。
 - **"DocsEditor"** - 具有页面编辑的PDF查看器，包括插入/删除页面、提取页面、重新排序页面等。
 - **"TestFile"** - 包含测试文件的文件夹。
 - **"lib"** - 一个包含 **"compdfkit-x.x.x.jar"** 的文件夹。
compdfkit-x.x.x.jar 是PDF核心API。
- **"api_reference_java"** - API 参考文档。
- **"developer_guide_java.pdf"** - 开发者指南。
- **"release_notes.txt"** - 发布信息。
- **"legal.txt"** - 法律和版权信息。



2.3 应用许可证密钥

ComPDFKit PDF SDK 是一个商业 SDK，需要许可证才能授权开发者开发或发布其应用程序。在开发模式下，每个许可证都绑定着唯一的设备 ID。ComPDFKit 拥有灵活的授权模式，请联系[我们的销售团队](#)了解更多信息。然而，即使拥有了许可证，也禁止将 ComPDFKit 的任何文件、示例代码或源代码给任何第三方。

离线许可证： 在安全性较高、无法连接到互联网或离线环境的场景下，提供离线许可证选项。离线许可证允许在无法连接到互联网的情况下进行授权和使用 ComPDFKit PDF SDK。

2.3.1 获取许可证秘钥

如果您打算将ComPDFKit许可证用于您的应用程序，我们提供两种类型的许可证：试用许可证和正式许可证。正式许可证需要绑定到应用程序的唯一 `deviceId`。ComPDFKit提供了一种获取许可证的方法。

方法:

1. 通过在ComPDFKit官网填写[联系销售](#)页面的需求表格与销售团队建立联系。
2. 收到您提交的需求后，我们的销售团队将在24小时内与您联系，并明确您的需求。
3. 在确认需求内容无误后，您将获得相应的免费试用许可证，有效期为30天，这期间您遇到的任何使用问题，我们都提供免费的技术支持。
4. 对产品满意后，您可选择购买正式许可证，交易完成后，我们的销售团队会以邮件的形式发送正式的许可证给您。

2.3.2 复制许可证秘钥

正确获取许可证密钥，是许可证应用的关键。

1. 在您收到的邮件中，找到包含许可证密钥的 `XML` 文件
2. 打开 `XML` 文件.

离线许可证:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<license version="1">
  <platform>linux</platform>
  <starttime>xxxxxxx</starttime>
  <endtime>xxxxxxx</endtime>
  <key>LICENSE_KEY</key>
</license>
```

3. 复制 `<key>LICENSE_KEY</key>` 字段中的**LICENSE_KEY**

2.3.3 Apply the License Key

您可以[联系ComPDFKit团队](#)获取试用许可证。在使用任何 ComPDFKit PDF SDK 类之前，必须执行以下操作将许可证用于应用：

1. 打开你的项目。
2. 根据您的要求，通过上一步获得的离线身份验证来初始化许可证。
3. 初始化许可证：
 - 离线认证:

```
NativeKMPDFKit.initialize("LICENSE_KEY", "your deviceId", "", Constants.LIB_DIR_NAME);
```

2.4 如何运行Demo

ComPDFKit PDF SDK for Java为开发人员提供了多个演示，以了解如何在Java应用程序上调用SDK。您可以在 **"samples"** 文件夹中找到它们。在本指南中，我们以 **"DocumentInfoTest"** 为例，展示如何在Java应用程序上运行它。

1. 在IDE上导入 **"samples"** 项目。
2. 导航到 **"DocumentInfoTest"** 文件中的 **"JAVA"** 文件夹，然后运行示例。例如

```
/samples/DocumentInfoTest/JAVA。
```

```
./RunTest.sh
```

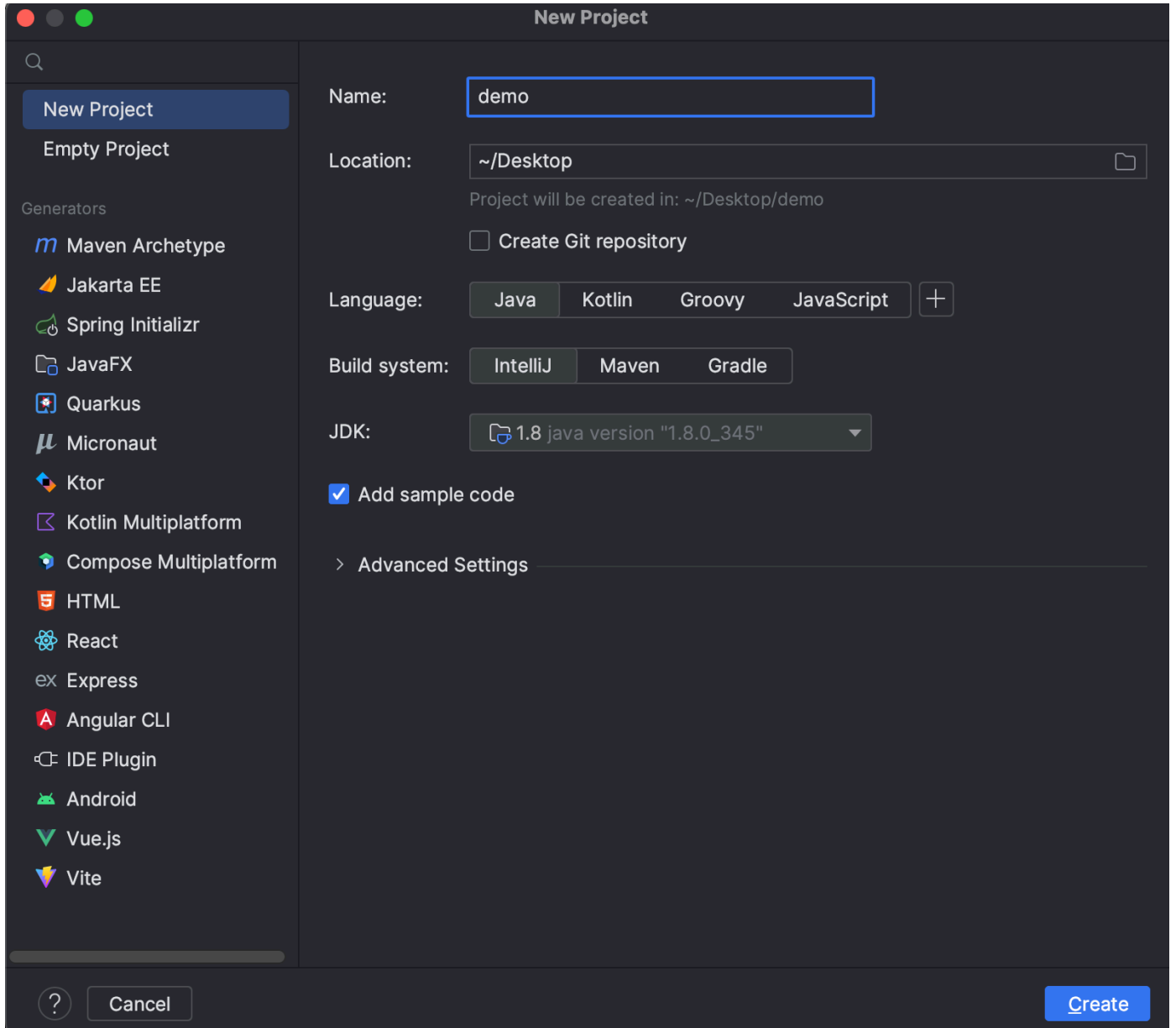
注: 这是一个演示项目，展示了已完成的ComPDFKit PDF SDK功能。根据您购买的许可证，功能可能会有所不同。请检查您选择的功能在这个演示项目中工作正常。输出文件将在 `/samples/out` 中。

2.5 如何在Java应用程序中使用ComPDFKit PDF SDK

本节将帮助您快速开始使用ComPDFKit PDF SDK，以制作带有分步说明的Java应用程序。

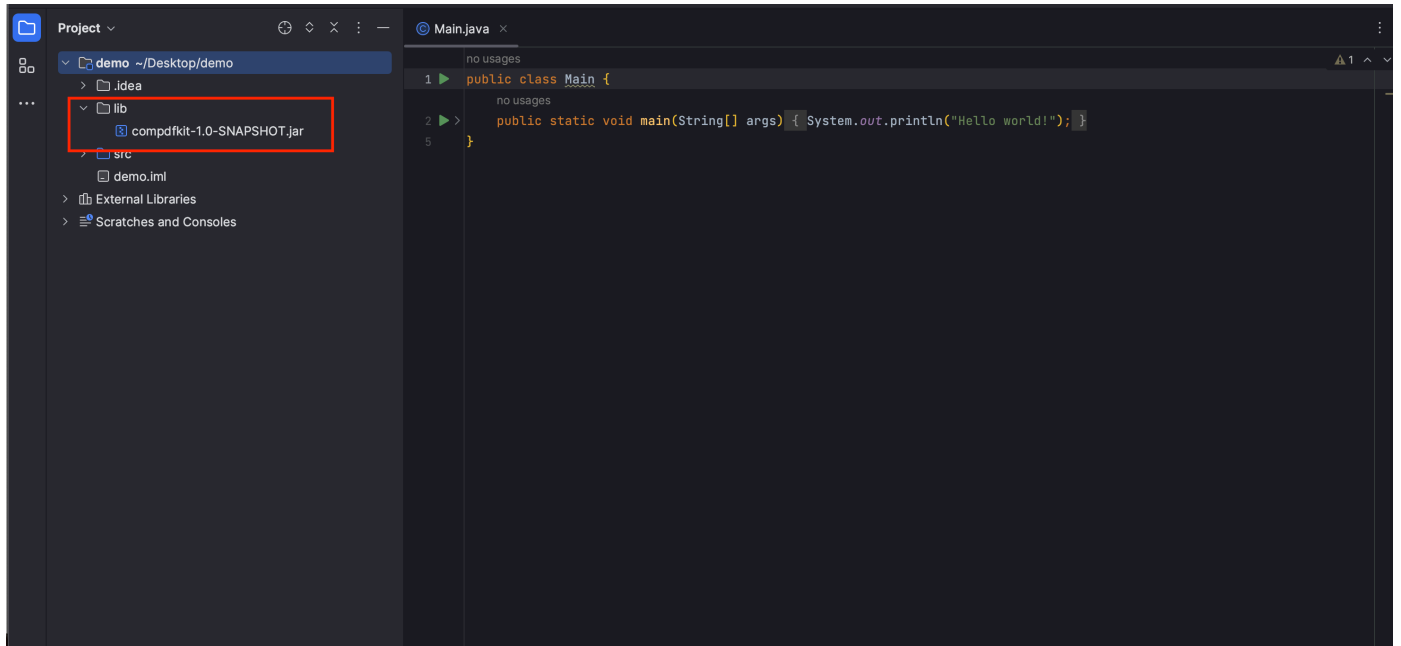
2.5.1 创建一个新项目

1. 使用IDE创建一个Java应用程序。在这里，我们创建了一个新的Java项目。

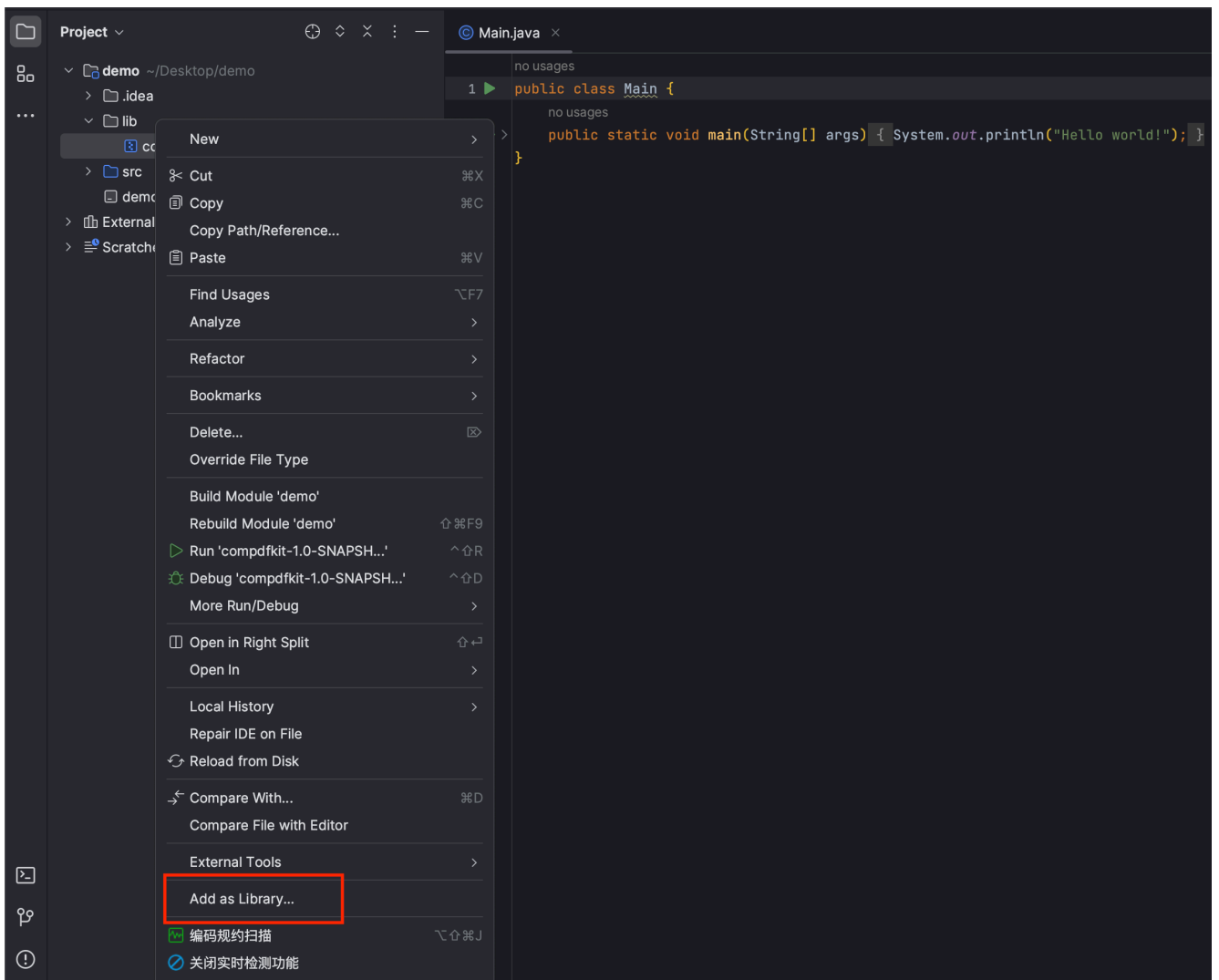


2.5.2 添加适用于Java的ComPDFKit PDF SDK软件包

1. 我们需要做的第一件事就是导入 ComPDFKit PDF SDK for Java。将“**lib**”复制到项目中。



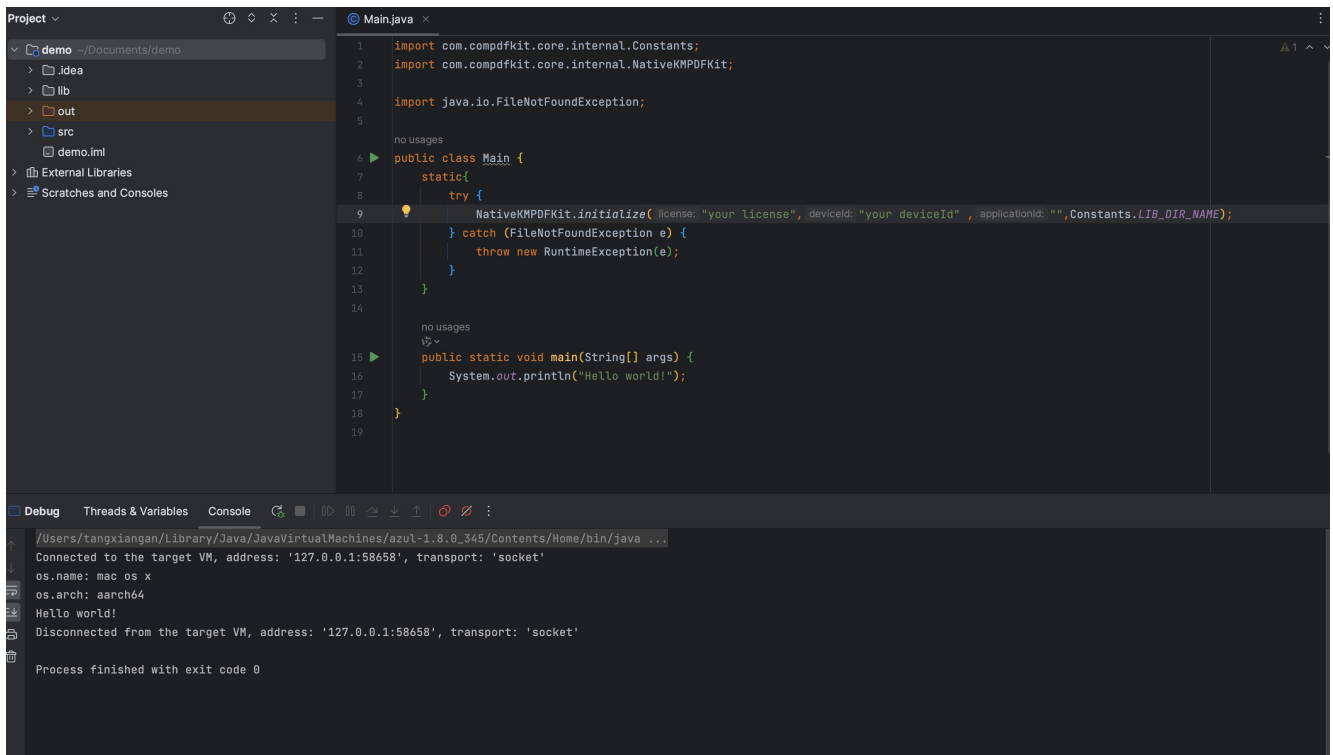
2. 右键单击SDK并选择 **Add as Library....**



3. 将以下代码添加到 **Main.java**:

```
static{
    try {
        NativeKMPDFKit.initialize("your license", "your deviceId", "",
Constants.LIB_DIR_NAME);
    } catch (FileNotFoundException e) {
        throw new RuntimeException(e);
    }
}
```

4. 启动打印计算机系统版本信息的 `main` 方法。如果信息打印成功，您已经成功集成了适用于Java的 ComPDFKit PDF SDK！



2.5.3 问题排除

1. 问题

如果您在集成 ComPDFKit PDF SDK for Java 时遇到其他问题，请随时[联系](#) ComPDFKit 团队。

2.6 Samples

Samples 使用预设参数、文档，和模块化的代码示例来调用 ComPDFKit PDF SDK 的每个功能的 API，无需 UI 交互或参数设置。这些功能包括创建/获取/删除各种类型的注释和表单、提取文本/图像、加密/解密文档、添加水印/贝茨码等。

这些项目不仅展示了每个功能的最佳实践，还提供了详细的介绍。可以在输出目录中观察到每个功能对 PDF 文档的影响。借助 Samples，你可以快速学习如何使用你需要的功能，并将其应用到你的项目中。

名称	描述
Annotation	打印注释列表信息、设置注释、和删除注释。注释包括标记类注释、便签、手绘、文字、圆、矩形、直线、图章等。
AnnotationImportExport	通过 XFDF 文件，导出/导入注释。
InteractiveForms	打印表单信息、创建、设置、删除、填写交互式表单（支持按钮、复选框、单选按钮、文本字段、下拉框、列表框和签名字段等）。
PDFPage	处理 PDF 页面，包括插入空白页面、插入 PDF 文档页面、拆分页面、合并页面、删除页面、旋转页面、替换文档页面和导出文档页面等。
DocumentInfo	提取 PDF 文档的信息，如：作者、创建日期。
Encrypt	加密/解密文档，对文档设置密码进行文档权限管控解密。
Watermark	添加/删除/设置/平铺文字或图片水印。
Compare	比较两个文档之间的差异，提供内容对比和覆盖对比。
PDFRedact	不可逆地进行密文标记，移除不可外泄的隐私或机密信息。

3 功能指南

如果您对概述部分提到的所有功能感兴趣，请参阅我们的指南，快速为您的应用程序添加 PDF 查看、注释和编辑功能。下面的部分列出了一些示例，展示了如何使用我们的 ComPDFKit PDF SDK API 向 Java 应用程序添加文档功能。

3.1 基本操作

3.1.1 介绍

处理文档时有一些常用的基础操作。

ComPDFKit 基本操作类型

- [打开文档](#)
打开本地文档，或创建全新 PDF 文档。
- [保存文档](#)
将文档保存到原路径，或将其另存为一份新文档。
- [文档信息](#)
查看文件创建者，创建时间，修改时间等。

3.1.2 打开文档

ComPDFKit 支持打开本地 PDF 文档或创建 PDF 文档。

打开本地 PDF 文档

以下是使用文件路径打开一个 PDF 文档的步骤：

1. 获取本地文件路径。
2. 使用文件路径初始化 `CPDFDocument` 对象。

以下是打开 PDF 文档的示例代码：

```
CPDFDocument document = new CPDFDocument();
// 打开文档。
CPDFDocument.PDFDocumentError error = document.open(pdfUri);
if (error == CPDFDocument.PDFDocumentError.PDFDocumentErrorPassword) {
    // 打开加密文档：使用密码打开文档。
    error = document.open(pdfUri, "password");
}
if (error == CPDFDocument.PDFDocumentError.PDFDocumentErrorSuccess) {
    // 文档已成功打开，并且可以对数据进行解析和操作。
} else {
    // 无法打开PDF文件。您可以参考API文件了解具体的错误消息。
}
// 检查文档在打开过程中是否已修复。
if (document.hasRepaired()) {
    // 修复后的文档需要先保存，然后再打开，否则无法保存对修复后文档所做的任何编辑或修改。
    // 执行保存操作。
    ...
}
```

创建 PDF 文档

以下是创建 PDF 文档的示例代码：

```
CPDFDocument document = CPDFDocument.createDocument();
```

默认创建的新文档不包含任何页面，需要参阅“文档编辑”功能了解如何创建新页面，以及向文档中加入现有页面的详细信息。

打开文档状态说明

以下是打开文档时的状态及其说明：

错误码	描述
PDFDocumentErrorSuccess	成功打开文档。
PDFDocumentErrorUnknown	未知错误。
PDFDocumentErrorFile	文件未找到或者不能被打开。
PDFDocumentErrorFormat	文件不是 PDF 格式或已损坏。
PDFDocumentErrorPassword	需要密码或密码不正确。
PDFDocumentErrorSecurity	不支持的安全方案。
PDFDocumentErrorPage	页面未找到或内容错误。
PDFDocumentNotVerifyLicense	许可证不允许该权限。
PDFDocumentErrorNoReadPermission	没有阅读权限

3.1.3 保存文档

ComPDFKit 支持增量保存与全量保存。

当文档保存路径为原文档路径时，则 PDF 文档将以增量方式保存，这意味着所有更改都会附加到文件中，在处理大型文件时，这可以明显加快文件保存速度，但是在每次保存文档时都将导致文档尺寸增加。

当文档保存路径为新文档时，则 PDF 文档将触发全量文档保存，这将重写整个文档，而不是在末尾追加更改。

以下是增量保存与全量保存的示例代码：

```
// 首先，查看文档是否已修改。如果有任何修改，请保存。
if (document.hasChanges()) {
    boolean res;
    // 将文档对象增量保存到当前路径
    res = document.save(CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
    // 全量保存到指定的路径；isRemoveSecurity指示是否删除文档的密码。
    //res = document.save(uri,
    CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveRemoveSecurity);
}
// 关闭文档并释放资源。
document.close();
```

3.1.4 文档信息

以下是获取文档信息的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFInfo info = document.getInfo();
String title = info.getTitle();           // 文档标题。
String author = info.getAuthor();         // 文档作者。
String subject = info.getSubject();       // 文档主题。
String creator = info.getCreator();       // 创建文档的应用程序名称。
String producer = info.getProducer();     // 生成 PDF 数据的应用程序名称。
String keywords = info.getKeywords();     // 文档关键字。
String creationDate = info.getCreationDate(); // 文档创建日期。
String modificationDate = info.getModificationDate(); // 文档最后修改日期。
```

3.2 注释

3.2.1 介绍

注释允许用户在不修改原作者内容的基础上，在 PDF 文档上划取段落、添加标记、评论、签署或盖章等，并将原文和注释内容一同进行共享。

ComPDFKit 注释的优势

- **全类型支持**：划词高亮，文字，手绘，形状，图章以及更多。
- **创建、编辑、删除**：通过编程方式或直接使用 UI 完成创建、编辑、删除操作。
- **注释扁平化**：将注释以图片方式永久添加到文档上，从而保证文档外观稳定且不可再更改。
- **注释事件**：触发指定的工作流程，实现自动化。
- **注释导入与导出**：将注释导出为 XFDF 模板并添加到多个文档。

ComPDFKit 注释的功能

- [获取注释](#)
取得文档中的注释列表和单个注释对象。
- [创建注释](#)
创建各种类型的注释。
- [编辑注释](#)
编辑注释位置，外观和属性。
- [更新注释外观](#)
修改注释属性后，更新注释外观以应用更改。
- [删除注释](#)
从文档中移除指定注释。
- [导入与导出](#)
将注释导出为 XFDF 模板并添加到多个文档。
- [注释扁平化](#)
将文档现有注释以图片方式永久保存到文档中。

3.2.2 ComPDFKit 注释支持的类型

ComPDFKit 支持符合 PDF 标准的注释类型（如 PDF 参考中所定义的那样），可以由符合标准的 PDF 处理程序读取和写入，包括 Adobe Acrobat，支持的注释类型如下：

类型	描述	类名
便签注释	便签注释表现为小型图标或标签，用户点击后可以展开显示相关注释内容，这种注释类型用于添加个人笔记、提醒或备注。	CPDFTextAnnotation
链接注释	链接注释使用户能够直接跳转到文档中的其他位置或外部资源，提供更丰富的导航体验。	CPDFLinkAnnotation
文本注释	文本注释允许用户在 PDF 文档中插入自由格式的文字，用于添加注解、评论或解释文档内容。	CPDFFreetextAnnotation
图形：矩形，圆形，线条，箭头	包括矩形、圆形、线条和箭头等形状，用于在文档中绘制图形以突出或标记特定区域。	CPDFSquareAnnotation, CPDFCircleAnnotation, CPDFLineAnnotation
标记：高亮，下划线，删除线，波浪线	在 PDF 文档中添加标记，以突出、强调或说明特定内容，例如重要的段落、行或单词、关键词或表格等。	CPDFHighlightAnnotation, CPDFUnderlineAnnotation, CPDFStrikeoutAnnotation, CPDFSquigglyAnnotation
图章	图章注释是一种在 PDF 文档中插图章或印章的工具，可包含文本、图像或自定义设计，类似于在纸质文档上盖章的过程。	CPDFStampAnnotation

3.2.3 获取注释

获取注释列表和注释对象的步骤如下：

1. 获取页面对象。
2. 取得页面对象中的注释列表。
3. 遍历列表中的每一个注释对象。

以下是对应的代码：

```
CPDFDocument document = new CPDFDocument();
document.open("fileUrl");
int pageCount = document.getPageCount();
if (pageCount > 0){
    for (int pageIndex = 0; pageIndex < pageCount; pageIndex++) {
        List<CPDFAnnotation> annotations =
document.pageAtIndex(pageIndex).getAnnotations();
        if (annotations != null && !annotations.isEmpty() ){
            for (CPDFAnnotation annotation:annotations){
```

```

        if (annotation == null || !annotation.isValid()){
            continue;
        }
        // 开始对获取到的注释进行相关的操作。
    }
}
}
}
}

```

3.2.4 创建注释

ComPDFKit 支持全类型的注释，包括便签，链接，图形，标记，图章，手绘，音频注释等，满足多样的注释需求。

注意： 使用函数 `CPDFPage.convertRectToPage` 将Android设备坐标转换为PDF页面坐标；使用函数 `CPDFPage.convertPointFromPage` 将PDF页面坐标转换为Android设备坐标

创建便签注释

便签注释表现为小型图标或标签，用户点击后可以展开显示相关注释内容，这种注释类型用于添加个人笔记、提醒或备注，使用户能够在文档中添加个性化的附加信息，而不影响原始文本的可读性。

创建便签的步骤如下：

1. 获取需要创建注释的页面对象。
2. 在该页面上创建便签注释对象。
3. 设置注释属性。
4. 更新注释外观使其显示在文档上。

创建便签的代码如下：

```

// 插入页码。
int pageNumber = 0;
// 获取pdf的页面实例。
CPDFPage page = document.pageAtIndex(pageNumber);
CPDFTextAnnotation textAnnotation = (CPDFTextAnnotation)
page.addAnnot(CPDFAnnotation.Type.TEXT);
// 获取要插入的页面的实际大小。
RectF pageSize = page.getSize();
RectF insertRect = new RectF(0, 0, 50, 50);
textAnnotation.setColor(Color.RED);
textAnnotation.setAlpha(255);
textAnnotation.setRect(insertRect);
textAnnotation.setContent("ComPDFKit");
textAnnotation.updateAp();

```

创建链接注释

链接注释使用户能够直接跳转到文档中的其他位置或外部资源，提供更丰富的导航体验。

创建链接注释的步骤如下：

1. 获取需要创建注释的页面对象。
2. 在该页面上创建链接注释对象。
3. 通过 `CPDFDestination` 设置链接跳转到第2页。
4. 设置注释属性，并将 `CPDFDestination` 对象附加到注释上。

创建链接的代码如下：

```
// 插入页码。
int pageNumber = 1;
// 获取pdf的页面实例。
CPDFPage page = document.pageAtIndex(pageNumber);
CPDFLinkAnnotation linkAnnotation = (CPDFLinkAnnotation)
page.addAnnot(CPDFAnnotation.Type.LINK);
RectF insertRect = new RectF(0,0,100,100);
linkAnnotation.setRect(insertRect);

// 跳转到指定页码
CPDFGoToAction goToAction = new CPDFGoToAction();
CPDFDestination destination = new CPDFDestination(pageNumber,0,pageSize.height(),1f);
goToAction.setDestination(document, destination);
linkAnnotation.setLinkAction(goToAction);

// 打开指定网页
CPDFUriAction uriAction = new CPDFUriAction();
uriAction.setUri("website");
linkAnnotation.setLinkAction(uriAction);
// 将注释更新到文档上。
linkAnnotation.updateAp();
```

创建文本注释

文本注释允许用户在 PDF 文档中插入自由格式的文字，用于添加注解、评论或解释文档内容。

创建文本注释的步骤如下：

1. 获取需要创建注释的页面对象。
2. 在该页面上创建文本注释对象。
3. 设置注释属性。
4. 更新注释外观使其显示在文档上。

创建文本注释的代码如下：

```
// 插入页码。
int pageNumber = 0;
// 获取pdf的页面实例。
CPDFPage page = document.pageAtIndex(pageNumber);
CPDFFreeTextAnnotation freetextAnnotation = (CPDFFreeTextAnnotation)
page.addAnnot(CPDFAnnotation.Type.FREETEXT, document);
RectF insertRect = new RectF(0,0,100,100);
freetextAnnotation.setRect(insertRect);
```

```
// 设置注释属性。
freetextAnnotation.setContent("Hello World!");
freetextAnnotation.setFreetextAlignment(CPDFFreetextAnnotation.Alignment.ALIGNMENT_LEFT);
CPDFTextAttribute textAttribute = freetextAnnotation.getFreetextDa();
textAttribute.setColor(Color.YELLOW.getRGB());
textAttribute.setFontSize(14);
textAttribute.setFontName("fontName");
freetextAnnotation.setFreetextDa(textAttribute);
freetextAnnotation.updateAp();
// 将注释更新到文档上。
freetextAnnotation.updateAp();
```

创建图形注释

图形注释包括矩形、圆形、线条和箭头等形状，用于在文档中绘制图形以突出或标记特定区域，表达文字不易描述的信息。

创建图形注释的步骤如下：

1. 获取需要创建注释的页面对象。
2. 在该页面上依次创建矩形，圆形，线段注释对象。
3. 设置注释属性。
4. 依次更新注释外观使其显示在文档上。

创建图形注释的代码如下：

```
// 插入页码。
int pageNumber = 0;
// 获取pdf的页面实例。
CPDFPage page = document.pageAtIndex(pageNumber);
// 在该页面上创建矩形注释对象。
CPDFSquareAnnotation squareAnnotation = (CPDFSquareAnnotation)
page.addAnnot(CPDFAnnotation.Type.SQUARE, document);
RectF insertRect = new RectF(0,0,100,100);
squareAnnotation.setRect(insertRect);
// 设置注释属性。
squareAnnotation.setBorderColor(Color.YELLOW.getRGB());
CPDFBorderStyle borderStyle = new CPDFBorderStyle(CPDFBorderStyle.Style.Border_Solid,
10, new float[]{8.0F, 0F});
borderStyle.setBorderWidth(10F);
squareAnnotation.setBorderStyle(borderStyle);
squareAnnotation.setBorderAlpha(255);
squareAnnotation.setFillColor(Color.RED.getRGB());
squareAnnotation.setFillAlpha(255);
// 将注释更新到文档上。
squareAnnotation.updateAp();

// 在该页面上创建圆形注释对象。
```



```

CPDFCircleAnnotation circleAnnotation = (CPDFCircleAnnotation)
page.addAnnot(CPDFAnnotation.Type.CIRCLE, document);
RectF insertRect = new RectF(0,0,100,100);
circleAnnotation.setRect(insertRect);
// 设置注释属性。
circleAnnotation.setBorderColor(Color.YELLOW.getRGB());
CPDFBorderStyle borderStyle = new CPDFBorderStyle(CPDFBorderStyle.Style.Border_Solid,
10, new float[]{8.0F, 0F});
borderStyle.setBorderWidth(10F);
circleAnnotation.setBorderStyle(borderStyle);
circleAnnotation.setBorderAlpha(255);
circleAnnotation.setFillColor(Color.RED.getRGB());
circleAnnotation.setFillAlpha(255);
// 将注释更新到文档上。
circleAnnotation.updateAp();

// 在该页面上创建线段注释对象。
CPDFLineAnnotation lineAnnotation = (CPDFLineAnnotation)
page.addAnnot(CPDFAnnotation.Type.LINE, document);
float lineWidth = 10f;
PointF startPoint = new PointF(0,0);
PointF endPoint = new PointF(200,200);
RectF area = new RectF(0,0,200,200);
lineAnnotation.setRect(area);
lineAnnotation.setLinePoints(startPoint, endPoint);
// 设置注释属性。
lineAnnotation.setLineType(CPDFLineAnnotation.LineType.LINETYPE_NONE,
CPDFLineAnnotation.LineType.LINETYPE_ARROW);
CPDFBorderStyle borderStyle = new
CPDFBorderStyle(CPDFBorderStyle.Style.Border_Solid,lineWidth,null);
lineAnnotation.setBorderStyle(borderStyle);
lineAnnotation.setBorderAlpha(255);
lineAnnotation.setBorderColor(Color.RED.getRGB())

// 将注释更新到文档上。
lineAnnptation.updateAp();

```

线段注释类型枚举

名称	描述
LINETYPE_UNKNOWN	非标准或无效的线段端点。
LINETYPE_NONE	没有线段端点。
LINETYPE_ARROW	两条短线以尖锐的角度相交，形成一个开放的箭头。
LINETYPE_CLOSEDARROW	两条短线以尖锐的角度相交，就像样式一样，并通过第三条线连接，形成一个三角形闭合的箭头，内部填充了注释的内部颜色。
LINETYPE_SQUARE	一个填充有注释内部颜色的正方形。
LINETYPE_CIRCLE	一个填充有注释内部颜色的圆形。
LINETYPE_DIAMOND	一个填充有注释内部颜色的菱形形状。
LINETYPE_BUTT	一条与线本身垂直的短线，位于端点上。
LINETYPE_ROPENARROW	两条与相反方向的短线。
LINETYPE_RCLOSEDARROW	一个与相反方向的三角形闭合箭头。
LINETYPE_SLASH	一条短线，位于端点上，大约比与线本身垂直的方向顺时针旋转30度。

创建标记注释

在 PDF 文档中添加标记，以突出、强调或说明特定内容，例如重要的段落、行或单词、关键词或表格等。ComPDFKit 提供高亮，下划线，波浪线，删除线四种标记注释。

创建标记注释的步骤如下：

- 1. 获取需要创建注释的页面对象。
- 2. 通过页面对象创建文本对象。
- 3. 使用该文本对象取得需要添加标记的文本位置。
- 4. 在该页面上创建对应的标记对象。
- 5. 设置标记对象的属性。
- 6. 更新注释外观使其显示在文档上。

以高亮注释为例，创建标记注释的代码如下：

```
// 首先，从页面上的特定区域获取TextSelection数组，选择文本。
CPDFPage pdfPage = douc.pageAtIndex(1);
CPDFTextPage pdfTextPage = pdfPage.getTextPage();
RectF selectRect = new RectF(0f, 0f, 500f, 500f);
// 然后，为特定区域添加高亮显示注释。
highlightAnnotation.setQuadRects(selectRect);
highlightAnnotation.setMarkedText("text");
highlightAnnotation.setRect(annotRect);
highlightAnnotation.updateAp();
```

创建图章注释

图章注释用于标识和验证文档的来源和真实性，ComPDFKit 支持标准图章，文字图章，图像图章。

创建图章注释的步骤如下：

1. 获取需要创建注释的页面对象。
2. 在该页面上创建对应的图章。
3. 设置图章属性。
4. 更新注释外观使其显示在文档上。

创建图章注释的代码如下：

```
// 获取需要创建注释的页面对象。
int pageNumber = 0;
CPDFPage pdfPage = document.pageAtIndex(pageNumber);

// 创建标准印章。
int yOffset = 50;
float lastOffset = 0;
for (int i = 0; i < CPDFStampAnnotation.StandardStamp.values().length; i++) {
    CPDFPage page = document.pageAtIndex(4);
    CPDFStampAnnotation.StandardStamp standardStamp =
CPDFStampAnnotation.StandardStamp.values()[i];
    if (standardStamp == null || standardStamp ==
CPDFStampAnnotation.StandardStamp.UNKNOWN) {
        continue;
    }
    CPDFStampAnnotation standard = (CPDFStampAnnotation)
page.addAnnot(CPDFAnnotation.Type.STAMP, document);
    if (standard == null) {
        continue;
    }
    standard.setStandardStamp(standardStamp);
    RectF pageSize = page.getSize();
    RectF insertRect = standard.getRect();
    float defaultWidth = 100F;
    int x = 50;
    if (i == 10) {
        lastOffset = 50;
    }
    if (i >= 10) {
        x = 150;
    }
    yOffset = (int) lastOffset + 10;
    PointF vertex = new PointF(x, yOffset);
    insertRect.set(vertex.x, vertex.y, vertex.x + defaultWidth, vertex.y + defaultWidth
* Math.abs(insertRect.height() / insertRect.width()));
    lastOffset = insertRect.bottom;
    standard.setRect(insertRect);
    standard.updateAp();
}
```

// 创建文字印章。

```
CPDFPage page = document.pageAtIndex(4);
CPDFStampAnnotation stampAnnotation = (CPDFStampAnnotation)
page.addAnnot(CPDFAnnotation.Type.STAMP, document);
SimpleDateFormat df = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
String date = df.format(new Date());
stampAnnotation.setTextStamp(new CPDFStampAnnotation.TextStamp(
    "ComPDFKit", date, CPDFStampAnnotation.TextStampShape.TEXTSTAMP_RECT.id,
    CPDFStampAnnotation.TextStampColor.TEXTSTAMP_GREEN.id));
RectF insertRect = stampAnnotation.getRect();
insertRect.set( insertRect);
float defaultWidth = 150f;
PointF vertex = new PointF(300, 50);
insertRect.set(vertex.x, vertex.y, vertex.x + defaultWidth, vertex.y + defaultWidth *
Math.abs(insertRect.height() / insertRect.width()));
stampAnnotation.setRect(insertRect);
stampAnnotation.updateAp();
```

// 创建图片印章。

```
CPDFStampAnnotation standard = (CPDFStampAnnotation)
page.addAnnot(CPDFAnnotation.Type.STAMP, document);
String imagePath = rootDir + "/TestFiles/ComPDFKit.png";
RectF imageInsertRect = new RectF(457, 626.5f, 513, 570.5f);
standard.setRect(imageInsertRect);
if (imagePath.endsWith(".png")) {
    File imageFile = new File(imagePath);
    BufferedImage image = null;
    try {
        image = ImageIO.read(imageFile);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    int[] pixel = getPixel(image);
    standard.updateApWithBitmap(pixel, image.getWidth(), image.getHeight());
} else {
    standard.setImageStamp(imagePath);
    standard.updateAp();
}
```

创建手绘注释

手绘注释直接快捷，用于直接绘制标注。

创建手绘注释的步骤如下：

1. 获取需要创建注释的页面对象。
2. 在该页面上创建手绘注释。
3. 设置手绘注释经过的路径。
4. 设置注释其他属性。

5. 更新注释外观使其显示在文档上。

创建手绘注释的代码如下：

```
// 获取需要创建注释的页面对象。
int pageNumber = 0;
CPDFPage page = document.pageAtIndex(pageNumber);
// 设置手绘注释经过的路径。
CPDFPage page1 = document.pageAtIndex(0);
ArrayList<ArrayList<PointF>> mDrawing = new ArrayList<>();
ArrayList<PointF> arc1 = new ArrayList<>();
arc1.add(new PointF(100, 100));
arc1.add(new PointF(110, 110));
arc1.add(new PointF(120, 120));
mDrawing.add(arc1);
ArrayList<PointF> arc2 = new ArrayList<>();
arc2.add(new PointF(115, 115));
arc2.add(new PointF(130, 130));
arc2.add(new PointF(160, 160));
mDrawing.add(arc2);
float scaleValue = 1F;
float borderWidth = 5F;
// 在该页面上创建手绘注释。
CPDFInkAnnotation inkAnnotation = (CPDFInkAnnotation)
page1.addAnnot(CPDFInkAnnotation.Type.INK, document);
inkAnnotation.setColor(Color.RED.getRGB());
inkAnnotation.setAlpha(255);
inkAnnotation.setBorderWidth(borderWidth);
RectF rect = null;
RectF size = document.getPageSize(0);
if (size.isEmpty()) {
    return;
}
int lineCount = mDrawing.size();
PointF[][] path = new PointF[lineCount][];
for (int lineIndex = 0; lineIndex < lineCount; lineIndex++) {
    ArrayList<PointF> line = mDrawing.get(lineIndex);
    int pointCount = line.size();
    PointF[] linePath = new PointF[pointCount];
    for (int pointIndex = 0; pointIndex < pointCount; pointIndex++) {
        PointF point = line.get(pointIndex);
        /***** 计算路径所包围的最小矩形 *****/
        if (rect == null) {
            rect = new RectF(point.x, point.y, point.x, point.y);
        } else {
            rect.union(point.x, point.y);
        }
    }
    /***** 计算转换为页面并存储在linePath集合中的坐标点 *****/
    linePath[pointIndex] = point;
}
```

```

    }
    path[lineIndex] = linePath;
}
float dx = borderWidth / scaleValue / 2;
rect.inset(-dx, -dx);
rect.set(rect);
inkAnnotation.setInkPath(path);
inkAnnotation.setRect(rect);
inkAnnotation.updateAp();

```

创建回复注释

回复注释用于对注释进行评论回复。

创建回复注释的步骤如下：

1. 获取需要创建回复的页面对象和注释。
2. 在该页面注释上创建回复注释。
3. 设置注释其他属性。

创建回复注释的代码如下：

```

// 创建回复注释
CPDFPage page = document.pageAtIndex(0);
CPDFFreetextAnnotation freetextAnnotation = (CPDFFreetextAnnotation)
page.getAnnotations().get(0);      CPDFAnnotation replyAnnotation =
freetextAnnotation.createReplyAnnotation();
if (replyAnnotation.isReplyAnnot()) {
    Date date = new Date();
    CPDFDate cpdfDate = new CPDFDate(date.getYear(), date.getMonth(), date.getDate(),
date.getHours(), date.getMinutes(), date.getSeconds(), 480);
    replyAnnotation.setCreationDate(cpdfDate);
    replyAnnotation.setRecentlyModifyDate(cpdfDate);
    replyAnnotation.setTitle("Guest");
    replyAnnotation.setContent("ComPDFKit Reply");
}

```

3.2.5 编辑注释

编辑注释的步骤如下：

1. 获取需要编辑注释的页面对象。
2. 获取到该页面的注释列表。
3. 在注释列表中寻找想要编辑的注释，并将其转换为合适的子类型。
4. 将属性设置到注释对象中。
5. 更新注释外观使其显示在文档上。

以下是编辑文本注释属性的代码：

```

CPDFPage page = document.pageAtIndex(0);
List<CPDFAnnotation> annotations = page.getAnnotations();
if (annotations != null && annotations.size() > 0) {
    CPDFAnnotation annotation = annotations.get(0);
    if (annotation.getType() == CPDFAnnotation.Type.FREETEXT) {
        CPDFFreetextAnnotation freetextAnnotation = (CPDFFreetextAnnotation) annotation;
        CPDFTextAttribute textAttribute = new CPDFTextAttribute("fontname", 20,
Color.YELLOW.getRGB());
        freetextAnnotation.setFreetextDa(textAttribute);
        freetextAnnotation.updateAp();
    }
}
}

```

3.2.6 更新注释外观

注释可能包含描述其外观的属性，例如注释颜色或形状。但是，这些并不能保证注释在不同的PDF阅读器中显示相同。为了解决这个问题，每个注释可以定义一个应用于渲染注释的外观流。

当您修改注释属性时，必须调用 `CPDFAnnotation` 类中的 `updateAp()` 方法。

```

public boolean updateAp();

```

设置注释的自定义外观流非常容易。通常在图章注释中会进行这一操作，因为它们没有其他属性。以这种方式使用的图章注释通常被称为图像注释。

3.2.7 删除注释

删除注释的步骤如下：

1. 获取需要删除注释的页面对象。
2. 获取该页面的注释列表。
3. 在注释列表中寻找想要删除的注释。
4. 删除该注释。

删除注释的代码如下：

```

// 获取需要删除注释的页面对象。
CPDFPage page = document.pageAtIndex(0);
// 获取该页面的注释列表。
List<CPDFAnnotation> annotations = page.getAnnotations();
if (annotations != null && annotations.size() > 0) {
    // 在注释列表中寻找想要删除的注释并将其移除。
    page.deleteAnnotation(annotations.get(0));
}
}

```

3.2.8 导入和导出注释

导入和导出 XFDF 注释的方法使用户能够在不改变原始 PDF 文档的情况下保存和还原注释及表单数据，方便在不同的编辑器或平台之间共享和处理文档。

导入注释

在通过 XFDF 导入注释时，会产生临时文件目录，在导入注释时需要同时指定 xfdf 路径和临时文件路径。

导入注释的代码如下：

```
//the path of xfdf
String xfdfPath = "/xfdf/exportXFDF.xfdf";
//the transfer path of audio and video files
String tmpxfdfPath = context.getCacheDir().getAbsolutePath() + "/tmp";
document.importAnnotations(xfdfPath,tmpxfdfPath);
```

导出注释

在通过 XFDF 导出注释时，会产生临时文件目录，在导出注释时需要同时指定 xfdf 路径和临时文件路径。

导出注释的代码如下：

```
//the path of xfdf
String xfdfPath = "/xfdf/exportXFDF.xfdf";
//the transfer path of audio and video files
String tmpxfdfPath = context.getCacheDir().getAbsolutePath() + "/tmp";
document.exportAnnotations(xfdfPath,tmpxfdfPath);
```

什么是XFDF

XFDF（XML Forms Data Format）是一种用于描述和传输 PDF 表单数据的XML格式。它通常与 PDF 文件一起使用，用于存储和传递表单字段的值、状态和操作。

XFDF 文件包含了对应 PDF 表单的数据，其中包括表单字段的名称、值、选项、格式等。

XFDF 是一种用于描述表单数据的格式，并不包含 PDF 文件本身。它用于存储和传输表单数据，以便在不同系统和应用程序之间进行交互和共享。

3.2.9 注释扁平化

注释扁平化是指将可编辑的注释转换为不可编辑、不可修改的静态图像或纯文本形式。当对注释进行扁平化时，整个文档所有可编辑元素（包含注释和表单）都将执行扁平化，所以注释扁平化又称为文档扁平化。

以下是注释扁平化的示例代码：


```
// 从文件路径打开文档。
CPDFDocument document = new CPDFDocument();
document.open(pdfPath, passwod);
// 展平文档的所有页面。
boolean result =
document.flattenAllPages(CPDFPage.PDFFlattenOption.FLAT_NORMALDISPLAY);
// 保存文档。
if (result) {
    document.save(CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveIncremental);
}
```

什么是文档扁平化？

文档扁平化是指将一个包含注释、表单字段或图层的 PDF 文档中的这些可编辑元素转换为不可编辑、不可修改的静态图像或纯文本形式的过程。这个过程的目的将文档的最终状态锁定，使其不再包含可编辑的元素。

文档扁平化通常在以下情境下应用：

1. **保护文档内容：** 扁平化可用于保护文档内容，确保文档在传播或分享过程中不被更改。这对于确保文档的完整性和保密性非常重要。
2. **表单提交：** 在表单处理中，扁平化可以用于将用户填写的表单字段和注释转换为静态图像，以便方便传输、存档或打印，同时防止在后续阶段对表单内容的修改。
3. **兼容性和显示：** 一些 PDF 阅读器或浏览器对于包含许多注释或图层的 PDF 文档的显示和交互可能存在问题。文档扁平化可以帮助解决这些兼容性问题，提高文档在各种环境中的可视化效果。
4. **减小文件大小：** 扁平化后的文档通常会减小文件大小，因为可编辑元素被转换为静态图像或文本，不再需要保存编辑信息的附加数据。

3.3 表单

3.3.1 介绍

表单（或称为 AcroForm）功能允许用户在 PDF 文档中创建可交互的表单域，以便其他用户可以通过填写表单域来提供信息。从本质上讲，PDF 表单域是一种 PDF 注释，称为 Widget 注释，它们用于实现交互式表单元素，例如按钮、复选框、组合框等。

由于 PDF 是一种电子格式，因此为用户提供了一些纸质表单所没有的优势。例如，能够编辑已经输入过的信息。此外，对于文档创建者来说，他们还能在互联网上分发 PDF 表单，限制填写表单的用户所输入的内容和格式，以及通过编程方式自动提取和分类用户填写的内容。

ComPDFKit 表单的优势

- **全类型支持：** 支持所有符合 PDF 规范的表单域类型、属性和外观设置。
- **创建、编辑、删除表单域：** 通过编程方式或直接使用 UI 完成创建、编辑、删除表单域操作。
- **填写表单域：** 通过 `CPDFReaderView` 的表单填写功能，或通过编程方式自动填写。
- **表单事件：** 触发指定的工作流程，实现自动化。
- **表单扁平化：** 将表单以图像方式永久添加到文档上，从而保证文档外观稳定且不可再更改。

ComPDFKit 表单的功能

- [创建表单域](#)

创建各种可交互表单域。

- [填写表单域](#)

在表单域中添加内容。

- [编辑表单域](#)

编辑已添加表单域的外观和内容。

- [删除表单域](#)

删除指定的表单域。

- [表单扁平化](#)

扁平化表单，使得不可以被编辑。

3.3.2 表单域类型说明

ComPDFKit 支持多种符合 PDF 标准的表单域类型，可以由许多程序读取和写入，包括 Adobe Acrobat 和其他符合标准的 PDF 处理程序，支持的表单域类型如下：

类型	描述	类名
Check Box(复选框)	从预定义选项中选择一个或多个选项	CPDFCheckboxWidget
Radio Button(单选按钮)	从预定义选项中选择一个选项	CPDFRadiobuttonWidget
Push Button(按钮)	在文档上创建自定义按钮，按下按钮时执行操作	CPDFPushbuttonWidget
List Box(列表框)	从预定义选项列表选择一个或多个选项	CPDFListboxWidget
Combo Box(下拉菜单)	从可用文本选项的下拉列表选择一个选项	CPDFComboboxWidget
Text(文本域)	输入文本内容，如姓名，地址，电子邮箱等	CPDFTextWidget
Signature(签名域)	对 PDF 文档进行数字签名或电子签名	CPDFSignatureWidget

3.3.3 创建表单域

创建文本域

文本域允许用户在指定区域输入文本，常用于收集用户信息或填写表格。

创建文本域的步骤如下：

1. 从 CPDFDocument 上获取到需要创建文本域的页面对象。
2. 在页面对象上创建文本域。
3. 设置文本域的位置和其它属性。

以下是创建文本域的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
int pageNumber = 0;
CPDFPage page = document.pageAtIndex(pageNumber);
```

```

RectF pageSize = document.getPageSize(pageNumber);
RectF singleLineTextRect = new RectF(28, 32, 237, 75);
//Insert a single-line TextField.
CPDFTextWidget textField = (CPDFTextWidget)
page.addFormWidget(CPDFWidget.WidgetType.Widget_TextField,document);
textField.setRect(singleLineTextRect);
textField.setFieldName("TextField1");
textField.setText("Basic Text Field");
textField.setFontColor(Color.BLACK.getRGB());
textField.setFontSize(15);
textField.setFillColor(Color.RED.getRGB());
textField.setBorderColor(Color.YELLOW.getRGB());
textField.setBorderWidth(5F);
textField.updateAp();

```

创建按钮

按钮允许用户在 PDF 页面上执行操作，如页面跳转和超链接跳转。

创建按钮的步骤如下：

1. 从 CPDFDocument 中获取页面对象。
2. 在页面对象上创建按钮。
3. 设置按钮位置，外观属性。
4. 创建并设置按钮的文本属性。
5. 创建并设置按钮的行为。
6. 更新按钮外观。

以下是创建按钮的示例代码：

```

CPDFDocument document = CPDFDocument();
document.open(pdfPath);
RectF pageSize = document.getPageSize(0);
CPDFPage cpdfPage = document.pageAtIndex(0);

//创建按钮用于跳转到第二页
RectF pushButton1Rect = new RectF(267F, 203F, 401F, 235F);
CPDFPushButtonWidget pushButtonWidget1 = (CPDFPushButtonWidget)
cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_PushButton,document);
pushButtonWidget1.setRect(pushButton1Rect);
pushButtonWidget1.setFieldName("PushButton1");
pushButtonWidget1.setFontColor(Color.BLACK.getRGB());
pushButtonWidget1.setFontSize(15);
pushButtonWidget1.setButtonText("PushButton");
CPDFGoToAction goToAction = new CPDFGoToAction();
CPDFDestination destination = new CPDFDestination(1, 0, 842, 0F);
goToAction.setDestination(document, destination);
pushButtonWidget1.setButtonAction(goToAction);
pushButtonWidget1.updateAp();

```

//创建按钮用于跳转到网页

```
RectF pushButton2Rect = new RectF(433F, 203F, 567F, 235F);
CPDFPushButtonWidget pushButtonWidget2 = (CPDFPushButtonWidget)
cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_PushButton,document);
pushButtonWidget2.setRect(pushButton2Rect);
pushButtonWidget2.setFieldName("PushButton2");
pushButtonWidget2.setFontColor(Color.BLACK.getRGB());
pushButtonWidget2.setFontSize(15);
pushButtonWidget2.setButtonTitle("PushButton");
CPDFUriAction uriAction = new CPDFUriAction();
uriAction.setUri("https://www.compdpf.com/");
pushButtonWidget2.setButtonAction(uriAction);
pushButtonWidget2.updateAp();
```

创建列表框

列表框允许用户从预定义的选项列表选择一个或多个项目，提供便捷的数据选择功能。

创建列表框的步骤如下：

1. 从 CPDFDocument 上获取到需要创建列表框的页面对象。
2. 在页面对象上创建列表框。
3. 设置列表框位置。
4. 在列表框中添加列表项。
5. 设置列表框属性。

以下是创建列表框的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF listBoxRect = new RectF(267F, 32F, 567F, 138F);
CPDFListboxWidget listBoxWidget = (CPDFListboxWidget)
cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_ListBox,document);
listBoxWidget.setRect(listBoxRect);
listBoxWidget.setFieldName("ListBox1");
listBoxWidget.setFillColor(Color.WHITE.getRGB());
listBoxWidget.setBorderColor(Color.BLACK.getRGB());
listBoxWidget.setBorderWidth(2);
CPDFWidgetItem[] listBoxItems = new CPDFWidgetItem[]{
    new CPDFWidgetItem("List Box No.1", "List Box No.1"),
    new CPDFWidgetItem("List Box No.2", "List Box No.2"),
    new CPDFWidgetItem("List Box No.3", "List Box No.3"),
};
listBoxWidget.setOptionItems(listBoxItems);
listBoxWidget.setSelectedIndexes(new int[]{1});
listBoxWidget.updateAp();
```

创建签名域

签名域允许用户在文档中插入数字签名或电子签名，用于验证文档的真实性和完整性。

创建签名域的步骤如下：

1. 从 CPDFDocument 上获取需要添加签名域的页面对象。
2. 在页面对象上创建签名域。
3. 设置签名域位置。
4. 设置签名域属性。

以下是创建签名域的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF signatureRect = new RectF(28, 206, 237, 301);
CPDFSignatureWidget signatureWidget = (CPDFSignatureWidget)
cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_SignatureFields,document);
signatureWidget.setFieldName("Signature1");
signatureWidget.setFill(Color.WHITE.getRGB());
signatureWidget.setBorderColor(Color.BLACK.getRGB());
signatureWidget.setBorderWidth(2F);
signatureWidget.setRect(signatureRect);
signatureWidget.updateAp();
```

创建复选框

复选框允许用户通过勾选或取消勾选来表示某个选项的状态。

创建复选框的步骤如下：

1. 从 CPDFDocument 中获取需要添加复选框的页面对象。
2. 在页面对象上创建复选框。
3. 设置复选框位置。
4. 设置复选框属性。

以下是创建复选框的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF checkBoxRect = new RectF(267, 251, 299, 283);
CPDFCheckboxWidget checkboxWidget = (CPDFCheckboxWidget)
cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_CheckBox,document);
checkboxWidget.setRect(checkBox1);
checkboxWidget.setFieldName("CheckBox1");
```

```
checkboxWidget.setFill(Color.parseColor("#CCE5E5FF").getRGB());
checkboxWidget.setBorderColor(Color.BLACK.getRGB());
checkboxWidget.setBorderWidth(2F);
checkboxWidget.setChecked(false);
checkboxWidget.updateAp();
```

创建单选框

单选框允许用户在一组预定义选项中选择唯一的单个选项。

创建单选框的步骤如下：

1. 从 CPDFDocument 中获取需要添加单选框的页面对象。
2. 在页面对象上创建单选框。
3. 设置单选框位置。
4. 设置单选框属性。

以下是创建单选框的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF radioButtonRect = new RectF(385F, 251F, 424F, 290F);
CPDFRadiobuttonWidget radiobuttonWidget = (CPDFRadiobuttonWidget)
cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_RadioButton, document);
radiobuttonWidget.setRect(radioButtonRect);
radiobuttonWidget.setFieldName("RadioButton");
radiobuttonWidget.setCheckStyle(CPDFWidget.CheckStyle.CK_Circle);
radiobuttonWidget.setFill(Color.decode("#CCE5E5FF").getRGB());
radiobuttonWidget.setBorderColor(Color.BLACK.getRGB());
radiobuttonWidget.setBorderWidth(2F);
radiobuttonWidget.setChecked(false);
radiobuttonWidget.setFieldName("RadioButton1");
radiobuttonWidget.updateAp();
```

创建下拉菜单

可以下拉的区域，选中下拉的一项将显示在该区域中。

创建下拉菜单的步骤如下：

1. 从 CPDFDocument 中获取需要添加下拉菜单的页面对象。
2. 在页面对象中创建下拉菜单。
3. 在下拉菜单中添加列表项。
4. 设置下拉菜单属性。

以下是创建下拉菜单的示例代码：

```
CPDFDocument document = new CPDFDocument();
```

```

document.open(pdfPath);
CPDFPage cpdfPage = document.pageAtIndex(0);
RectF pageSize = document.getPageSize(0);

RectF comboBoxRect = new RectF(267F, 143F, 567F, 189F);
CPDFComboboxWidget comboBoxWidget = (CPDFComboboxWidget)
cpdfPage.addFormWidget(CPDFWidget.WidgetType.Widget_ComboBox,document);
comboBoxWidget.setRect(comboBoxRect);
comboBoxWidget.setFieldName("ComboBox1");
CPDFWidgetItem[] comboBoxItems = new CPDFWidgetItem[]{
    new CPDFWidgetItem("Combo Box No.1", "Combo Box No.1"),
    new CPDFWidgetItem("Combo Box No.2", "Combo Box No.2"),
    new CPDFWidgetItem("Combo Box No.3", "Combo Box No.3"),
};
comboBoxWidget.setOptionItems(comboBoxItems, new int[]{1});
comboBoxWidget.updateAp();

```

3.3.4 填写表单域

使用代码获取 PDF 文档中的表单域并在表单域中填写内容。

由于表单是特殊的注释类型，继承自注释类。所以对注释的接口也同样适用于表单。

使用代码填写表单域的步骤如下：

1. 从 CPDFDocument 中获取想要填写表单的页面对象。
2. 获取页面对象中的所有注释。
3. 遍历所有注释，找到需要填写的表单。
4. 根据需求修改表单填写内容。

以下是填写表单域的示例代码：

```

CPDFDocument document = new CPDFDocument();
document.open(pdfPath);

for (int i = 0; i < document.getPageCount(); i++) {
    CPDFPage page = document.pageAtIndex(i);
    for (CPDFAnnotation annotation : page.getAnnotations()) {
        switch (annotation.getType()) {
            case WIDGET:
                CPDFWidget cpdfWidget = (CPDFWidget) annotation;
                switch (cpdfWidget.getWidgetType()) {
                    case Widget_TextField:
                        CPDFTextWidget textWidget = (CPDFTextWidget) cpdfWidget;
                        textWidget.setText("test");
                        textWidget.updateAp();
                        break;
                    case Widget_RadioButton:
                        CPDFRadiobuttonWidget radiobuttonWidget = (CPDFRadiobuttonWidget)
cpdfWidget;

```

```

        radiobuttonWidget.setChecked(true);
        radiobuttonWidget.updateAp();
        break;
    case Widget_ListBox:
        CPDFListboxWidget listBoxWidget = (CPDFListboxWidget) cpdfWidget;
        listBoxWidget.setSelectedIndexes(new int[]{0});
        listBoxWidget.updateAp();
        break;
    default:
        break;
}
break;
default:
break;
}
}
}
}

```

3.3.5 编辑表单域

获取并编辑已添加表单域的外观和内容。

需要注意，不同的表单域类型，允许设置的属性可能不完全一致。

编辑表单域的步骤如下：

1. 取得需要编辑的表单域对象。
2. 修改表单域属性。
3. 更新表单域外观。

以下是编辑表单域的代码：

```

CPDFTextWidget textWidget = (CPDFTextWidget) cpdfWidget;
textWidget.setFillColor(Color.RED.getRGB());
textWidget.updateAp();

```

3.3.6 删除表单域

由于表单继承自注释类，所以删除表单域和删除注释的方法一致。

删除表单域的步骤如下：

1. 从 CPDFDocument 中获取想要删除表单的页面对象。
2. 删除表单域。

以下是删除指定文档中第一个注释的示例代码：


```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage page = document.pageAtIndex(0);

List<CPDFAnnotation> annotationList = page.getAnnotations();
annotationList.remove(0);
```

3.3.7 表单扁平化

表单扁平化是指将可编辑的表单域转换为不可编辑、不可修改的静态图像或纯文本形式。当对表单域进行扁平化时，整个文档所有可编辑元素（包含注释和表单）都将执行扁平化，所以表单扁平化又称为文档扁平化。

以下是表单扁平化的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
document.flattenAllPages(CPDFPage.PDFFlattenOption.FLAT_NORMALDISPLAY);
document.save("savePath", isRemoveSecurity);
```

什么是文档扁平化？

文档扁平化是指将一个包含注释、表单字段或图层的 PDF 文档中的这些可编辑元素转换为不可编辑、不可修改的静态图像或纯文本形式的过程。这个过程的目的是将文档的最终状态锁定，使其不再包含可编辑的元素。

文档扁平化通常在以下情境下应用：

1. **保护文档内容：** 扁平化可用于保护文档内容，确保文档在传播或分享过程中不被更改。这对于确保文档的完整性和保密性非常重要。
2. **表单提交：** 在表单处理中，扁平化可以用于将用户填写的表单字段和注释转换为静态图像，以便方便传输、存档或打印，同时防止在后续阶段对表单内容的修改。
3. **兼容性和显示：** 一些 PDF 阅读器或浏览器对于包含许多注释或图层的 PDF 文档的显示和交互可能存在问题。文档扁平化可以帮助解决这些兼容性问题，提高文档在各种环境中的可视化效果。
4. **减小文件大小：** 扁平化后的文档通常会减小文件大小，因为可编辑元素被转换为静态图像或文本，不再需要保存编辑信息的附加数据。

3.4 文档编辑

3.4.1 介绍

文档编辑功能提供了一系列的操作页面的能力，使用户能够控制文档结构，并调整文档的布局和格式，确保文档内容以合理有序的方式精准呈现。

ComPDFKit 文档编辑的优势

- **插入或删除页面：** 向文档插入或删除页面，以满足特定的排版要求。
- **文档合并与拆分：** 将多个文档或页面合并成新文档，或将大文档拆分成多个小文档。
- **文档结构调整：** 调整页面排列顺序或旋转方向，以满足特定的显示或打印需求。
- **多文档协作：** 从一个文档中提取页面，然后插入到另一个文档中，以促进协作和内容整合。
- **快速集成 UI：** 通过可扩展 UI 组件实现快速集成与自定义。

ComPDFKit 文档编辑的功能

- [插入页面](#)

向目标文档中插入空白页面，图片，或另一个文档中的页面。

- [拆分页面](#)

将多页文档的一部分拆分成独立文档。

- [合并页面](#)

将多个文档的页面合并为一个文档。

- [删除页面](#)

删除文档中的页面。

- [旋转页面](#)

旋转 PDF 文档中的页面。

- [替换页面](#)

将目标文档中的指定页面用另一个文档的页面替换。

- [提取页面](#)

提取文档中的页面。

3.4.2 插入页面

插入空白页面，或插入其他 PDF 页面到目标页面中。

插入空白页面

以下是插入空白页面的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
int pageIndex = 1; // 插入到第一页之后。
float pageWidth = 595F;
float pageHeight = 842F;
document.insertBlankPage(pageIndex, pageWidth, pageHeight);
```

插入其他 PDF 页面

以下是插入其他 PDF 页面的示例代码：

```
CPDFDocument document2 = new CPDFDocument();
document2.open(pdfPath);
document.importPages(document2, new int[]{0}, 1);
```

3.4.3 拆分页面

以下是拆分页面的步骤：

1. 为每个需要拆分的部分新建一个 `CPDFDocument` 对象。
2. 将目标文档需要被拆分的部分添加到新建的 `CPDFDocument` 对象中。

以下是拆分页面的示例代码：

```
CPDFDocument documentPart1 = CPDFDocument.createDocument();
documentPart1.importPages(document, new int[]{0,1}, 0); // 将第一页和第二页拆分到第一个部分。

CPDFDocument documentPart2 = CPDFDocument.createDocument();
documentPart2.importPages(document, new int[]{2,3,4}, 0); // 将第三，四，五页拆分到第二个部分。
```

3.4.4 合并页面

以下是合并页面的步骤：

1. 创建一个空白 PDF 文档。
2. 打开包含要合并页面的 PDF 文档。
3. 选中所有需要合并的页面，将其合并到同一个文档中。

以下是合并页面的示例代码：

```
CPDFDocument documentForMerge = CPDFDocument.createDocument();

CPDFDocument document1 = new CPDFDocument();
document1.open(pdfFile);

CPDFDocument document2 = new CPDFDocument();
document2.open(pdfFile);

documentForMerge.importPages(document1, 0, document1.getPageCount() - 1,
document.getPageCount());
documentForMerge.importPages(document2, 0, document2.getPageCount() - 1,
document.getPageCount());
```

3.4.5 删除页面

以下是删除页面的示例代码：

```
document.removePages(new int[]{0});
```

3.4.6 旋转页面

以下是旋转页面的示例代码：

```
document.pageAtIndex(0).setRotation(90); // 将第一页顺时针旋转90度。
```

3.4.7 替换页面

以下是替换页面的步骤：

1. 删除目标文件需要被替换的页码。
2. 将替换页面插入原文档被删除的位置，完成页面的替换。

以下是替换页面的示例代码：

```
// 删除文档的第一页。
document.removePages(new int[]{0});
CPDFDocument document2 = new CPDFDocument();
document2.open(pdfPath);
// 将另一份文档的第一页插入原文档的第一页位置，完成替换。
document.importPages(document2, new int[]{0}, 0);
```

3.4.8 提取页面

以下是提取页面的示例代码：

```
CPDFDocument extractDocument = CPDFDocument.createDocument();
// 将原文档的第一页提取到新建文档
extractDocument.importPages(document, new int[]{0}, 0);
```

3.5 安全

3.5.1 介绍

安全模块提供密码与权限设置、贝茨码、背景和页眉页脚功能，通过文档访问与权限控制，添加标识和版权信息，全面确保文档安全性。

ComPDFKit 安全功能的优势

- **权限控制：**通过设置与文档关联的安全权限，限制用户访问、复制、或打印等敏感权限。
- **密码管理：**设置、修改、或删除文档密码与权限。
- **加密标准：**支持标准 PDF 安全处理程序（40 和 128 位 RC4 加密），以及 128 和 256 位 AES（高级加密标准）加密。

ComPDFKit 安全模块的功能

- [PDF 权限](#)

通过管理文档密码和权限设置，防止未经授权的访问，并控制用户对文档的操作权限。

3.5.2 PDF 权限

PDF 权限模块用于确保 PDF 文档的安全性，提供加密、文档权限、解密、移除密码功能，以保障用户对文档的安全控制和有效管理。

加密

加密功能分为用户密码和所有者密码两部分。用户密码用于打开文档，确保只有授权的用户可以访问文档内容。当设置用户密码时，通常会限制一些诸如修改、复制或打印等文档权限。而所有者密码不仅可以打开文档，还能解锁所有被禁止的权限，使用户能够对文档进行修改、复制或打印等操作。双层密码体系旨在提供更加灵活和安全的文档访问和管理方式。

ComPDFKit 提供多种加密算法和权限设置，根据需求使用合适的算法，设置自定义权限以保护文档。

以下是加密的步骤：

1. 设定不同的用户密码和所有者密码。
2. 创建权限信息类。
3. 使用用户密码和所有者密码，权限信息，算法对文档加密。
4. 保存文档

以下是加密 PDF 文档的示例代码：

```
// Open document from file path.
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
// Set user password.
document.setUserPassword(user_password);
// Set owner password.
document.setOwnerPassword(owner_password);
// Set permission info.
CPDFDocumentPermissionInfo info = document.getPermissionsInfo();
info.setAllowsPrinting(false);
info.setAllowsCopying(true);
document.setPermissionsInfo(info);
// Save document.
document.save(CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
```

文档权限

PDF 规范中，支持对文档的多种权限进行设置，通过设置这些权限，可以限制用户只能做出符合预期的行为。

PDF 规范定义了如下所示的权限：

- 打印 - 打印文档。
- 高质量打印 - 高保真打印文档。
- 复制 - 复制文档内容。
- 文档更改 - 修改文档内容，但不包括文档属性。
- 文档组合 - 插入、删除和旋转页面。
- 注释 - 创建或修改文档注释，包括表单字段条目。
- 表单字段输入 - 修改表单字段条目，即使不能编辑文档注释。

以下是查看文档权限的步骤：

1. 获取文档权限信息。
2. 通过文档权限信息查看指定的权限。

以下是查看文档权限的示例代码：

```
// Open document from file path.
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
// get permission info.
CPDFDocumentPermissionInfo info = document.getPermissionsInfo();
System.out.println("Allows Printing:" + info.isAllowsPrinting());
System.out.println("Allows Copying:" + info.isAllowsCopying());
```

解密

访问具有密码保护的 PDF 文档，需要输入密码。ComPDFKit 提供了两种密码：用户密码和所有者密码。用户密码用于打开文档，确保只有授权的用户可以访问文档内容。而所有者密码不仅打开文档，还解锁所有权限，使用户能够对文档进行修改、打印或复制等操作。双层密码体系旨在提供更加灵活和安全的文档访问和管理方式。

使用不同级别的密码获取不同的权限级别。

解密的步骤如下：

1. 打开文档时判断文档是否已被加密。
2. 对于已加密的文档，输入用户密码或所有者密码均可打开文档。

以下是解密 PDF 文档的示例代码：

```
// Open document from file path.
CPDFDocument document = new CPDFDocument();
CPDFDocument.PDFDocumentError error = document.open(pdfPath);
if (error == CPDFDocument.PDFDocumentError.PDFDocumentErrorPassword) {
    // Password required.
    document.open(pdfPath, password);
}
```

删除密码

删除密码是指用户在已经获得了所有者权限的情况下，将文档的所有者密码和用户密码移除并另存为新的文档，新的文档将不再需要密码即可打开且默认可使用所有权限。

删除密码的步骤如下：

1. 解锁文档并获取所有权限。
2. 将解锁后的文档保存。

以下是删除密码的示例代码：

```
// Open document from file path.
CPDFDocument document = new CPDFDocument();
document.open(pdfPath, password);
// Remove password.
try {
    document.save(CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveRemoveSecurity);
} catch (CPDFDocumentException e) {
    e.printStackTrace();
}
```

3.6 标记密文

3.6.1 介绍

标记密文是一种有选择性地从文档中删除可见文本或图像的过程，通常会永久删除或覆盖敏感信息，并将其从 PDF 元数据中彻底清除。经修订的内容通常会用黑条或其他视觉上醒目的标记替代，以指示该信息已被删除。

标记密文通常用于保护隐私、遵守法律要求，或确保敏感内容不会被意外披露，以便安全分发给法院、专利和政府机构、媒体、客户、供应商或任何其他访问内容受到限制的受众。

ComPDFKit 标记密文的优势

- **文本加密：** 使用颜色块或特定字符覆盖敏感文本，使其变得不可读。
- **图像加密：** 使用颜色块或特定字符模糊或删除图像的部分以消除敏感细节。
- **永久变更：** 标记密文是永久性的，以确保敏感信息无法恢复。
- **元数据清除：** 除了对可见内容的删除，也会删除 PDF 中任何可能包含敏感信息的隐藏元数据。

ComPDFKit 标记密文的功能

- [创建标记密文](#)

在文档中创建标记密文以抹除特定信息。

3.6.2 创建标记密文

标记密文有以下两个步骤：

1. **创建密文注释：** 在需要标记的区域创建密文注释。这一步骤只是标记要进行应用密文更改的区域，在应用密文更改前可以设置，移动或删除这些密文注释，且不会从文档中删除内容。
2. **应用密文更改：** 标记好需要彻底删除内容的区域后，应用密文更改，密文注释所在区域的内容将被不可逆地删除。

通过这两个步骤，您可以创建密文注释并应用标记密文，彻底删除文档中的敏感数据。

创建密文注释

您可以通过 `CPDFRedactAnnotation` 类来创建密文注释。使用 `setRect` 方法设置应该被密文注释覆盖的区域。

此外，在此阶段还可以对密文的外观进行自定义设置。需要注意，一旦密文注释被应用，其外观将无法更改，因为密文注释将在应用后变为不可编辑、不可修改的静态内容，而非可交互的密文注释。

以下是创建密文注释的示例代码：

```
CPDFDocument document = new CPDFDocument();
document.open(pdfPath);
CPDFPage pdfPage = document.pageAtIndex(0);
CPDFRedactAnnotation redactAnnotation = (CPDFRedactAnnotation)
pdfPage.addAnnot(CPDFAnnotation.Type.REDACT,document);
RectF insertRect = new RectF(0, 0, 100, 100);
redactAnnotation.setRect(insertRect);
CPDFTextAttribute textAttribute = new CPDFTextAttribute("Helvetica", 14,
Color.YELLOW.getRGB());
redactAnnotation.setTextDa(textAttribute);
redactAnnotation.setFillColor(Color.RED.getRGB());
redactAnnotation.setOverLayText("REDACTED");
redactAnnotation.updateAp();
redactAnnotation.applyRedaction();
```

应用密文更改

ComPDFKit SDK 确保如果文本、图像或矢量图形包含在密文注释所标记的区域中，则该部分的图像或路径数据将被彻底删除，且无法还原。

以下是应用密文更改的示例代码：

```
redactAnnotation.applyRedactions();
```

3.7 水印

3.7.1 介绍

水印是放置在 PDF 文档上的标记，通常情况下是添加到 PDF 文档中的半透明文本或者图片元素。通过添加水印，可以在不干扰阅读的情况下突出显示文档所属对象的相关信息。

ComPDFKit 水印的优势

- **版权标识：**通过水印显示文档来源和版权信息，以此防止截屏或盗用。
- **形象展示：**自定义水印外观，展示品牌 logo 或其他预设内容。

ComPDFKit 水印的功能

- [添加文本水印](#)
创建自定义内容和外观的文本水印。
- [添加图片水印](#)
创建由自定义图片生成的图片水印。
- [删除水印](#)
删除附加在 PDF 文档中的水印。

3.7.2 添加文本水印

添加文字水印的步骤如下：

1. 初始化 CPDFWatermark 对象，指定水印类型为文本。
2. 设置文字水印需要的属性，包含内容，字体，颜色，字号。
3. 设置水印通用属性。
4. 在文档中创建水印。

以下是创建文字水印的示例代码：

```
CPDFWatermark watermark = (CPDFWatermark)
document.createWatermark(CPDFWatermark.Type.WATERMARK_TYPE_TEXT);
watermark.setText("Hello World");// 水印的文本（图像水印无效）。
watermark.setFontName("Helvetica");// 水印的文本字体（图像水印无效）。
watermark.setTextRGBColor(Color.decode(color).getRGB());// 水印的文本颜色（图像水印无效）。
watermark.setFontSize(30);// 默认字体: Helvetica 24。
watermark.setOpacity(0.5f);// 水印的透明度，范围为0~1，默认为1。
watermark.setFront(true);// 将水印定位在内容前面。
watermark.setVerticalAlign(CPDFWatermark.VerticalAlign.WATERMARK_VERTICAL_CENTER);// 水印的垂直
对齐方式。
watermark.setHorizontalAlign(CPDFWatermark.HorizontalAlign.WATERMARK_HORIZONTAL_CENTER);// 水印的水
平对齐方式。
watermark.setVertOffset(0);// 相对于垂直位置的平移量。正数向下移动，负数向上移动。
watermark.setHorizOffset(0);// 相对于水平位置的平移量。正数向右移动，负数向左移动。
watermark.setScale(1.3f);// 水印的缩放比例，默认为1，如果是图片水印，则1代表图片的原始尺寸，如果是
文本水印，则1代表`textFont`字体大小。
watermark.setPages("3,4,5");// 设置水印页面范围3,4,5。
watermark.setFullScreen(true);// 设置水印平铺（图片水印无效）。
watermark.setHorizontalSpacing(100);// 设置平铺水印的左右间距。
watermark.setVerticalSpacing(100);// 设置平铺水印的垂直间距。
watermark.update();
watermark.release();
```

3.7.3 添加图片水印

添加图片水印的步骤如下：

1. 初始 CPDFWatermark 对象，指定水印类型为图片。
2. 根据图片文件创建 image，为图片水印设置图片源和缩放比例。
3. 设置水印通用属性。
4. 在文档中创建水印。

以下是添加图片水印的示例代码：

```
CPDFWatermark watermark = (CPDFWatermark)
document.createWatermark(CPDFWatermark.Type.WATERMARK_TYPE_IMG);
watermark.setImage(bitmap, 100, 200);// 设置水印的图像（文本水印无效）。
watermark.setOpacity(0.5f);// 水印的透明度，范围为0~1，默认为1。
```

```
watermark.setFront(true); // 将水印定位在内容前面。
watermark.setVerticalAlign(CPDFWatermark.VerticalAlign.WATERMARK_VERTICAL_ALIGN_CENTER); // 水印的垂直对齐方式。
watermark.setHorizontalAlign(CPDFWatermark.HorizontalAlign.WATERMARK_HORIZONTAL_ALIGN_CENTER); // 水印的水平对齐方式。
watermark.setVertOffset(0); // 相对于垂直位置的平移量。正数向下移动，负数向上移动。
watermark.setHorizOffset(0); // 相对于水平位置的平移量。正数向右移动，负数向左移动。
watermark.setScale(1.3f); // 水印的缩放比例，默认为1，如果是图片水印，则1代表图片的原始尺寸，如果是文本水印，则1代表`textFont`字体大小。
watermark.setPages("3,4,5"); // 设置水印页面范围3,4,5。
watermark.setFullScreen(true); // 设置水印平铺（图片水印无效）。
watermark.setHorizontalSpacing(100); // 设置平铺水印的左右间距。
watermark.setVerticalSpacing(100); // 设置平铺水印的垂直间距。
watermark.update();
watermark.release();
```

3.7.4 删除水印

删除水印的步骤如下：

对 `CPDFDocument` 对象调用 `releaseAllWatermarks()`，即可删除文档中所有水印。

以下是删除水印的示例代码：

```
//删除文档中所有水印。
document.releaseAllWatermarks();
```

3.8 文档对比

3.8.1 介绍

ComPDFKit 水印的功能

- [覆盖对比](#)

覆盖比较用于直观地比较不同文档的页面。它对施工计划、详细图纸以及其他需要精确放置的内容都很有帮助。

- [内容对比](#)

通过比较 PDF 文件的两个版本，快速确定更改内容。

3.8.2 覆盖对比

进行覆盖对比的步骤如下：

1. 打开对比的两个文档。
2. 创建 `CPDFCompareDrawings` 对象。
3. 设置对比的颜色等属性。
4. 调用 `compareDrawings.getDocGenerate()` 获取对比结果并且保存到文档。

以下是覆盖对比的示例代码：

```
CPDFDocument baseDocument = new CPDFDocument();
baseDocument.open(rootDir + "/TestFiles/comparefile.pdf");
CPDFDocument comDocument = new CPDFDocument();
comDocument.open(rootDir + "/TestFiles/comparefile1.pdf");
printDividingLine();

CPDFCompareDrawings compareDrawings = baseDocument.createCompareDrawings(comDocument);
compareDrawings.setBlendMode(4);
compareDrawings.setColorCompare(Color.decode("#FBBDBF").getRGB());
compareDrawings.setColorCompared(Color.decode("#93B9FD").getRGB());
compareDrawings.setFillAlphaCompare(50);
compareDrawings.setFillAlphaCompared(50);
compareDrawings.compareContent();
CPDFDocument document = compareDrawings.getDocGenerate();
document.save(rootDir + "/out/compareTestOverlay.pdf",
CPDFDocument.PDFDocumentSaveType.PDFDocumentSaveNoIncremental);
```

3.8.3 内容对比

进行内容对比的步骤如下：

1. 打开对比的两个文档。
2. 创建 `CPDFCompare` 对象。
3. 设置对比的颜色等属性。
4. 调用 `compare.doCompare()` 获取对比结果并且使用 `generateNewDoc()` 方法保存到文档。

以下是内容对比的示例代码：

```
CPDFDocument baseDocument = new CPDFDocument();
baseDocument.open(rootDir + "/TestFiles/comparefile.pdf");
CPDFDocument comDocument = new CPDFDocument();
comDocument.open(rootDir + "/TestFiles/comparefile1.pdf");
printDividingLine();

CPDFCompare compare = baseDocument.createCompare(baseDocument, comDocument);
compare.setReplaceResColor(Color.decode("#93B9FD").getRGB());
compare.setInsertResColor(Color.decode("#C0FFEC").getRGB());
CPDFCompareResults result = compare.doCompare(0, 0, CPDFCompare.CompType.COMPAREALL.id,
true);
compare.generateNewDoc(rootDir + "/out/compareTest.pdf");
```

4 支持

4.1 报告问题

感谢您对 ComPDFKit PDF SDK 的兴趣，它是一款易于使用且功能强大的开发解决方案，可将高质量 PDF 渲染功能集成到您的应用程序中。如果您在使用 ComPDFKit PDF SDK for Android 时遇到任何技术问题或错误问题，请将问题报告提交给 ComPDFKit 团队。以下更多信息将帮助我们解决您的问题：

- ComPDFKit PDF SDK 产品和版本。
- 您的操作系统和 IDE 版本。
- 问题的详细描述。
- 任何其他相关信息，例如错误屏幕截图。

4.2 联系信息

网站链接：

- 英文主页：<https://www.compdpf.com>
- 中文主页：<https://www.compdpf.com/zh-cn>
- 开发者文档：<https://www.compdpf.com/guides/pdf-sdk/java/overview>
- 示例代码：<https://www.compdpf.com/guides/pdf-sdk/java/examples>
- API 参考文档：<https://developers.compdpf.com/guides/pdf-sdk/java/api-reference/index>

联系ComPDFKit：

- 联系销售：<https://www.compdpf.com/contact-sales>
- 技术问题反馈至：<https://www.compdpf.com/support>
- 电子邮件（直接联系）：support@compdpf.com

感谢您！ ComPDFKit 团队